Resumen para el parcialito: Drivers en user space · Funciones variádicas · Threads

Programación I (ITBA). Teoría y práctica: GPIO por sysfs en la Raspberry Pi, `stdarg.h`, pthreads, mutex y semáforos.

1 · Drivers en user space (GPIO por sysfs)

Capas del sistema operativo

Aplicación → glibc → System Call Interface → Kernel / VFS → Device driver → HW

- **Kernel space:** es lo único que accede *directamente* al hardware, de forma organizada. Impide que el usuario lo use "de cualquier forma".
- **User space** (Ring 3): ahí corren las aplicaciones, controladas para que no dañen al SO ni a otros procesos. **No tocan el HW**.
- **System call:** la API del kernel, la única puerta de entrada desde user space (`exit, fork, read, write, open, close, waitpid`...). `syscall(SYS_getpid)` (llamada directa) ≡ `getpid()` (*wrapper* de glibc).
- **Device driver:** el código del kernel que habla con un dispositivo, a través de la Driver API: `module_init`, `file_operations: open / read / write / release`, `module_exit`.
- **sysfs** (montado en `/sys`): un **file system virtual** del kernel que exporta información de dispositivos y drivers al user space. "Todo es un archivo": escribir en un archivo de sysfs ejecuta código del driver.

GPIO por sysfs: el orden importa

#	Escribir...	...en el archivo
1	`"17"`	`/sys/class/gpio/export` (crea `gpio17/`)
2	`"out"` / `"in"`	`/sys/class/gpio/gpio17/direction`
3	`"1"` / `"0"`	`/sys/class/gpio/gpio17/value` (si es entrada, se **lee**)
4	`"17"`	`/sys/class/gpio/unexport` (solo al terminar)

- Otros archivos de `gpioXX/`: `active_low` (si vale 1, invierte la polaridad), `edge`, `uevent`, `power`.
- Consola: `echo "17" > /sys/class/gpio/export` · `cat /sys/class/gpio/gpio17/value` · `ls /sys/class/gpio/`
- Hace falta root: `sudo su`. Sin root: `echo "17" | sudo tee /sys/class/gpio/export`.
- Placa del TP: D0..D7 = GPIO **17, 4, 18, 23, 24, 25, 22, 27**. Se mapea con un array: `led_gpio[led]`.

fopen() vs open()

	`fopen()` (alto nivel)	`open()` (bajo nivel)
Qué es	Función de la librería de C	Wrapper de la syscall (n.º 5)
Buffer	Buffered I/O	Unbuffered
Velocidad	Más rápida: no cambia a KS en cada escritura	Más lenta: cambia US→KS siempre
Portable	Sí	No
Handle	`FILE *fp`	`int fd` (file descriptor)

Clave: con `fopen`, lo que escribís queda en un buffer de user space. Llega al kernel recién con `fflush()` o `fclose()`, así que **el LED no cambia hasta ese momento**. Por la misma razón, los errores del kernel (por ejemplo EBUSY) aparecen en el `fclose`.

Valores de retorno (el famoso `nWritten`)

`nWritten` es solo una variable `int` que guarda lo que devuelve la función de escritura, para chequear errores.

Función	OK	Error
`write(fd,buf,n)`	Cantidad de bytes escritos	-1
`fputs(s,fp)`	Un valor ≥ 0	EOF (-1)
`fputc(c,fp)`	El carácter escrito	EOF
`fopen`	Un `FILE*`	NULL

Escribir en un archivo de sysfs desde C

```c
// devuelve 0 si OK, -1 si error
int write_file(const char *ruta, const char *txt) {
    FILE *f = fopen(ruta, "w");
    if (f == NULL) return -1;
    int err = (fputs(txt, f) == EOF);
    err |= (fclose(f) == EOF);   // flush → kernel
    return err ? -1 : 0;
}

// export + dirección + valor del GPIO 17
write_file("/sys/class/gpio/export", "17");
write_file("/sys/class/gpio/gpio17/direction", "out");
write_file("/sys/class/gpio/gpio17/value", "1");

// ruta armada a partir de un número
char ruta[64];
snprintf(ruta, sizeof ruta,
         "/sys/class/gpio/gpio%d/value", gpio);

// leer una entrada
FILE *f = fopen("/sys/class/gpio/gpio22/value", "r");
int v = fgetc(f) - '0';  fclose(f);
```

- Al pin se le escribe **texto** (`"17"`), no el número 17.
- `"/sys/class/gpio" "/export"`: dos literales seguidos se concatenan en tiempo de compilación.
- `hardware.c`: `hw_init` (export y `out`), `hw_led_write(led, est)`, `hw_close` (apaga y hace unexport).

Errores típicos (`perror`)

`Permission denied`	Falta `sudo`
`No such file…` en `gpioXX/`	El pin no está exportado
`Device or resource busy`	El pin ya estaba exportado
`Invalid argument`	Número de GPIO inválido (kernel 6.6+: offset de 512)

RPi: desde la PC, `scp main.c pi@IP:~/` · en la RPi, `gcc -Wall *.c -o tp && sudo ./tp`

2 · Funciones con número variable de argumentos (variadic)

Herramientas (`#include <stdarg.h>`)

Nombre	Qué hace
`va_list`	Un **tipo de dato** (no una macro). Se usa como cualquier otra variable y se puede pasar a otras funciones.
`va_start(ap, ultimo)`	Inicializa la lista. El 2.º parámetro es el **último parámetro fijo**, el anterior a `...`
`va_arg(ap, tipo)`	Devuelve el **siguiente** argumento, interpretado como `tipo`. Es secuencial.
`va_end(ap)`	"Borra" la lista (pone el puntero en NULL). Siempre va antes del `return`.

- Prototipo: `double f(long n, int k, ...);`. Necesita **al menos un parámetro fijo** antes de `...`
- **¿Cómo sabe la función cuántos argumentos recibió?** No puede saberlo sola. Se lo tiene que indicar el que llama: (a) un parámetro con la **cantidad** (`getMin(n, ...)`); (b) un **centinela** al final (`-1`, `NULL`); (c) un **string de formato** (`printf`: la cantidad de `%`).
- **¿Por qué son macros?** `va_arg` recibe un **tipo** como argumento, cosa que una función no puede hacer. Además manipulan punteros al stack de la función que llama, y eso depende de la arquitectura. Se expanden inline:

```
typedef char* va_list;
#define va_start(ap,v) (ap=(va_list)&v+_INTSIZEOF(v))
#define va_arg(ap,t) (*(t*)((ap+=_INTSIZEOF(t))-_INTSIZEOF(t)))
#define va_end(ap)   (ap=(va_list)0)
```

Promoción de argumentos: en la parte `...`, `char` y `short` se promueven a `int`, y `float` a `double`. Por eso **nunca** se escribe `va_arg(ap, char)` ni `va_arg(ap, float)`: `char c = (char) va_arg(ap, int);` · `float x = (float) va_arg(ap, double);`
Si el tipo que pedís no coincide con el que se pasó (por ejemplo, se pasa `5` y se lee `double`), el resultado es basura (comportamiento indefinido).

- `printf(char *fmt, ...)` es variádica. Las versiones **v** (`vprintf(fmt, ap)`, `vfprintf`, `vsprintf`) reciben un `va_list` en lugar de `...`, y sirven para "reenviar" los argumentos.

Ejemplos tipo parcial

```
// (a) con cantidad: getMin(5, 4,88,4,77,2) → 2
int getMin(int n, ...) {
    va_list ap;
    va_start(ap, n);
    int min = va_arg(ap, int);
    for (int i = 1; i < n; i++) {
        int x = va_arg(ap, int);
        if (x < min) min = x;
    }
    va_end(ap);
    return min;
}

// (b) con centinela: promedio(2, 4, 6, -1) → 4.0
double promedio(int primero, ...) {
    va_list ap;
    va_start(ap, primero);
    double suma = 0;  int n = 0;
    for (int x = primero; x != -1; x = va_arg(ap, int)) {
        suma += x;  n++;
    }
    va_end(ap);
    return n ? suma / n : 0;
}

// (c) tipos distintos con formato: imprimir("dfc", 3, 2.5f, 'x'
void imprimir(const char *fmt, ...) {
    va_list ap;
    va_start(ap, fmt);
    for (; *fmt; fmt++)
        switch (*fmt) {
        case 'd': printf("%d ", va_arg(ap, int));       break;
        case 'f': printf("%f ", va_arg(ap, double));    break;
        case 'c': printf("%c ", (char)va_arg(ap, int)); break;
        }
    va_end(ap);
}

// reenviar a vprintf: error("pin %d: %s", 17, "busy")
void error(const char *msg, ...) {
    va_list ap;
    printf("%s, line %d: error: ", filename, lineno);
    va_start(ap, msg);
    vprintf(msg, ap);         // NO printf(msg, ap)
    va_end(ap);
    printf("\n");
}
```

`powerSummation(3, 2, 1.0f, 2.0f, 3.0f)` = $1^2+2^2+3^2$ = **14**. Lee con `va_arg(ap, double)` porque los `float` se promovieron a `double`. `va_start(ap, raiseTo)` usa `raiseTo` porque es el último parámetro fijo.

3 · Threads (pthreads)

Conceptos

- **Concurrencia:** varias tareas avanzan de forma *aparentemente* simultánea, intercalándose en **1 CPU**. **Paralelismo:** se ejecutan de forma *realmente* simultánea en **varias CPUs**.
- **Proceso:** una instancia de un programa. Tiene su **propia memoria**, asignada por el SO, y no intercambia datos con otros procesos salvo que use un mecanismo especial (IPC).
- **Thread:** una unidad de ejecución **dentro** de un proceso. Los threads **comparten la memoria del proceso** (globales, heap, archivos abiertos), pero cada uno tiene **su propio PC y SP** (stack propio, es decir, variables locales privadas). Son más rápidos porque el cambio de contexto entre threads no tiene el overhead del cambio entre procesos.
- **Pthreads:** librería POSIX de threads, multiplataforma. `#include <pthread.h>` y se compila con `gcc ... -pthread` (o `-lpthread`).

API

```
int pthread_create(pthread_t *th, const pthread_attr_t *attr,
                   void *(*start_routine)(void *), void *arg);
void pthread_exit(void *value_ptr);
int pthread_join(pthread_t th, void **value_ptr);
```

- `create`: `th` es el ID del thread (sale por referencia); `attr` = NULL toma los valores por defecto (stack, prioridad, joinable/detached); `start_routine` es la función que va a ejecutar; `arg` es **un solo** argumento, por referencia (NULL si no hay). Devuelve **0 si OK**.
- `join`: **bloquea** al thread que llama hasta que `th` termina, y recibe lo que el thread pasó a `pthread_exit`. Devuelve 0 si OK.
- `pthread_exit(ptr)`: al terminar, el thread **libera su stack**, así que `ptr` tiene que apuntar a una variable **global** o a memoria **dinámica (malloc)**, nunca a una local.
- Si `main` termina (`return`) sin hacer `join`, el proceso muere y con él todos sus threads.

```
void *tarea(void *arg) {
    long id = (long)arg;            // recuperar el argumento
    printf("soy el thread %ld\n", id);
    return NULL;                    // ≡ pthread_exit(NULL)
}
int main(void) {
    pthread_t th[3];
    for (long i = 0; i < 3; i++)
        if (pthread_create(&th[i], NULL, tarea, (void *)i) != 0)
            printf("no se pudo crear\n");
    for (int i = 0; i < 3; i++) pthread_join(th[i], NULL);
    return 0;
}
```

Trampa: pasar `&i` de la variable del `for`. Todos los threads apuntan a la **misma** `i`, que sigue cambiando. Hay que pasar el valor casteado `(void*)i`, o un puntero a un elemento distinto de un array (`&ids[i]`).

Ej.: producto escalar con threads

```
#define N 1000
#define T 4
double a[N], b[N], total = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;

void *dot(void *arg) {
    long r = (long)arg;
    double parcial = 0;             // local, privada
    for (int i = r*N/T; i < (r+1)*N/T; i++)
        parcial += a[i] * b[i];
    pthread_mutex_lock(&m);
    total += parcial;               // 1 sola vez por thread
    pthread_mutex_unlock(&m);
    return NULL;
}
// main: create con (void*)r, r = 0..T-1; luego join
```

Repartir el trabajo (matriz × vector)

Si `y = A·x` se reparte con 3 threads y m = 6, cada thread calcula 2 componentes de `y`: `y[i] = Σ_j A[i][j]·x[j]`. Las filas de un thread van de `rank*m/T` a `(rank+1)*m/T`. Como **cada thread escribe en una variable distinta**, **no hace falta sincronizar**.

Condición de carrera (race condition)

Si varios threads escriben la **misma** variable (por ejemplo `sum += x` en el cálculo de π), la operación **no es atómica**: es leer, sumar y escribir. Si dos threads se intercalan, se pierden actualizaciones. El código que accede a datos compartidos es la **sección crítica**, y hay que protegerla.

Mecanismo	Cómo funciona	Contra
Busy-waiting	`while(flag != my_rank);` sección crítica `flag = (flag+1) % T;` (round robin)	Gira consumiendo CPU sin hacer nada, y además fuerza un orden fijo entre los threads.
Mutex	`lock`: si está tomado, el thread se **bloquea** (no consume CPU). Tiene 2 estados (locked/unlocked) y el lock es **atómico**.	Solo un thread a la vez. El que hace lock es el que hace unlock.
Semáforo	Contador ≥ 0. `wait`: si vale > 0 lo decrementa, si vale 0 se bloquea. `post`: incrementa.	Sirve para contar recursos y para **señalizar u ordenar** threads.

Mutex

```
pthread_mutex_t lock;               // GLOBAL
pthread_mutex_init(&lock, NULL);    // 0 si OK
pthread_mutex_lock(&lock);
    /* sección crítica (corta) */
pthread_mutex_unlock(&lock);
pthread_mutex_destroy(&lock);       // tras los join
// alternativa: = PTHREAD_MUTEX_INITIALIZER
```

Mutex0 vs Mutex1: sin mutex, el thread 2 hace `counter += 1` mientras el 1 todavía trabaja, y la salida muestra *"Job 2 finished"* dos veces. Con el mutex alrededor de toda la función, los jobs se ejecutan de a uno: 1 started, 1 finished, 2 started, 2 finished.

Semáforos (`#include <semaphore.h>`)

```
sem_t s;
sem_init(&s, 0, valor); // pshared=0: mismo proceso
sem_wait(&s);           // s>0: s--; s==0: bloquea
sem_post(&s);           // s++ (despierta a uno)
```

- Se inicializa con la **cantidad de recursos libres**. Si el máximo es 1, es un **semáforo binario** (equivale a un mutex). Si se inicializa en **0**, sirve para **señalizar**: "esperá a que yo te avise".
- **Sema1.c:** `len1` y `len2` arrancan en 0. El lector hace `scanf`, `post(len1)` y `wait(len2)`; el escritor hace `wait(len1)`, imprime y `post(len2)`. **¿Hace falta `len2`?** Sí. Sin ese semáforo, el lector volvería al `scanf` y podría pisar `answer` antes de que el escritor lo imprima. `len2` es el "ya lo imprimí" (handshake).

Ej.: módulo Timer y millis()

```
static volatile unsigned int ticks;      // timer.c
static void *timer_th(void *p) {
    while (ticks > 0) { sleep(1); ticks--; }
    return NULL;
}
void Set_Timer(unsigned int delay) {     // segundos
    pthread_t t;  ticks = delay;
    pthread_create(&t, NULL, timer_th, NULL);
}
unsigned char Get_Timer_Status(void) {
    return ticks == 0 ? TIMEOUT : RUNNING;
}
```

`while(Get_Timer_Status()!=TIMEOUT);` **es bloqueante** (espera activa: el main no hace nada más mientras tanto). Para que no lo sea, se **consulta** el estado dentro del lazo principal, mientras se hacen otras

Truco: acumular en una variable **local** y entrar a la sección crítica una sola vez. Eso es mucho más rápido que hacer lock en cada iteración.

tareas: `if (Get_Timer_Status()==TIMEOUT) { accion(); Set_Timer(1); }`

```
static volatile unsigned long ms = 0;    // millis()
static void *tick(void *p) { while (1) { usleep(1000); ms++; } }
void setup(void) { pthread_t t; pthread_create(&t, NULL, tick, N
unsigned long millis(void) { return ms; }
void loop(void) {                        // no bloqueante
    static unsigned long t0 = 0;
    if (millis() - t0 >= 500) { t0 = millis(); toggle_led(); }
}
int main(void) { setup(); while (1) loop(); }
```

`volatile` le avisa al compilador que la variable puede cambiar desde otro thread, para que la vuelva a leer siempre. th_ex0 sigue la misma idea: un thread base de tiempo decrementa `TimerTick` cada 100 ms.

4 · Preguntas cortas típicas

¿Por qué una aplicación no puede escribir directo en el registro del GPIO? Porque corre en user space (Ring 3). El acceso al HW es exclusivo del kernel, y se pide a través de system calls.

¿Qué es sysfs? Un file system virtual del kernel, montado en `/sys`, que exporta dispositivos y drivers al user space como archivos.

¿Qué pasa si escribís `direction` antes de `export`? Falla, porque el directorio `gpioXX/` todavía no existe.

Usé `fputs("1", f)` y el LED no prendió. Falta `fflush(f)` o `fclose(f)`: el dato quedó en el buffer de user space.

¿Diferencia entre `syscall(SYS_write,…)` y `write(…)`? Ninguna en el resultado. `write` es el wrapper de glibc de la misma system call.

¿Por qué hay que hacer unexport al final? Para devolverle el pin al kernel. Si no se hace, el próximo export falla con EBUSY.

¿Cuál es el 2.º argumento de `va_start`? El último parámetro fijo, el anterior a `...`

¿Por qué `va_arg(ap, float)` está mal? Porque el `float` se promovió a `double` al pasar por `...`. Lo mismo pasa con `char`/`short`, que se promueven a `int`.

¿Qué comparten y qué no los threads de un proceso? Comparten las globales, el heap y los archivos abiertos. Cada uno tiene su propio stack (variables locales), PC y SP.

¿Proceso o thread, cuál es más liviano y por qué? El thread: comparte el espacio de memoria, así que el cambio de contexto es más barato.

¿Qué devuelve `pthread_create`? 0 si salió bien. Si falla, devuelve un código de error, que se puede imprimir con `strerror(err)`.

¿Por qué no devolver la dirección de una variable local con `pthread_exit`? Porque el stack del thread se libera al terminar. Hay que usar una global o `malloc`.

¿Busy-wait o mutex? El mutex: el thread que espera queda bloqueado y no consume CPU. Con busy-wait gira inútilmente.

¿Mutex o semáforo? El mutex da exclusión mutua: lo libera el mismo thread que lo tomó. El semáforo es un contador: un thread puede hacer `post` para despertar a otro, y por eso sirve para ordenar o señalizar.

¿Cuándo no hace falta sincronizar? Cuando cada thread solo escribe sus propias variables (matriz × vector) y los datos compartidos solo se leen.

¿Qué hace `pthread_join`? Bloquea al thread que lo llama hasta que el thread indicado termina, y recoge su valor de retorno.